Лабораторна робота №4

Основи визначення та використання функцій. Рекурсивні та анонімні функції. Замикання та декоратори функцій.

Мета роботи: опанувати основи роботи з функціями, у тому числі з рекурсивними та анонімними. Познайомитися із функціями-замиканнями та декораторами.

Теоретичні відомості

1. Функції.

Мова Python дозволяє програмісту створювати власні функції. Для цього використовується наступний синтаксис:

```
def <ім'я функції>([список аргументів]):
    оператор 1
    оператор 2
    …
    оператор N
```

Тут ім'я функції вигадується програмістом подібно до імен змінних і, через те, що функція зазвичай виконує якусь дію, то в ролі імені слід обирати дієслово, наприклад: go, show, get, set тощо.

Далі йде набір операторів, які утворюють тіло функції. Саме вони починають виконуватися після її виклику.

Давайте створимо найпростішу функцію, яка виводитиме «hello» у консоль:

```
def say_hello():
    print("hello")
```

Дивіться, ми придумали ім'я функції `say_hello`, записали порожні круглі дужки без аргументів і через двокрапку визначили тіло функції у вигляді конструкції `print("hello")`. Але це лише визначення функції. Самого виклику тут ще немає і якщо запустити програму, нічого не станеться.

Щоб викликати цю функцію, потрібно вказати її ім'я і наприкінці обов'язково поставити круглі дужки, навіть якщо ми не передаємо їй жодних аргументів:

```
say_hello()
```

Ці круглі дужки є оператором виклику функції. Тепер після запуску програми в консолі з'явиться повідомлення "hello".

Ім'я функції без круглих дужок – це фактично посилання на функцію (на спеціальний об'єкт у пам’яті):

```
print(type(say_hello)) # <class 'function'>
```

А якщо це посилання, то ми можемо виконати таку операцію:

```
f = say_hello
```

і тим самим визначити її синонім (alias) і викликати її вже через це нове ім'я:

```
f()
```

Як ми говорили на початку, функції, зазвичай, створюються для їх багаторазового виклику. Дійсно, ми тепер можемо нашу функцію викликати в будь-якому місці нашої програми необхідну кількість разів, наприклад, так:

```
say_hello ()
print("---------------")
say_hello ()
```

Тут буде вже два виклики цієї функції. До того ж, ми викликаємо функцію тільки після її визначення. Тобто, якщо її викликати на початку програми, то виникне помилка, адже функція ще не буде визначена на момент виклику.

Якщо потрібно визначити ще одну функцію, ми її можемо записати після першої:

```
def my_abs(x):
    x = -x if (x < 0) else x
```

Імена функцій мають бути унікальними (як і імена змінних). Наша нова функція обчислюватиме модуль переданого їй числа, тобто, на відміну від функції `say_hello`, наша функція `my_abs` приймає один аргумент. Відповідно, у круглих дужках зазначаємо ім’я змінної `x`, яка набуде значення переданого під час виклику числа. Якщо тепер ми її викличемо, то побачимо значення `None`:

```
print(my_abs(-5)        # None
```

Це сталося тому, що функція `my_abs` не повертає ніякого значення. Але нам хотілося б, щоб функція повертала значення змінної `x`, обчисленої в тілі функції. Для цього потрібно записати оператор `return`, після якого через пропуск вказуємо величину, що повертається:

```
def my_abs(x):
    x = -x if (x < 0) else x
    return x
```

Тепер при виклику функції отримаємо очікуване значення 5.

Такий підхід дозволяє передавати функції різні значення, наприклад, так:

```
print(my_abs(15))      # 15
a = -100
print(my_abs(a))       # 100
```

І це робить її роботу універсальною – з будь-якими числовими даними. Причому, коли під час виконання тіла функції інтерпретатор зустрічає оператор `return`, функція завершує свою роботу:

```
def my_abs(x):
    x = -x if (x < 0) else x
    return x
    print(x)
```

У разі виклику цієї функції оператор `print` не відпрацює, а, отже, в консоль нічого виводитись не буде. Якщо ж поставити `print` до оператора `return`, то після виклику ми побачимо значення змінної x у консолі:

```
def my_abs(x):
    x = -x if (x < 0) else x
    print(x)
    return x
```

Тіло функції може містити декілька операторів `return`. Функція завершить своє виконання, як тільки інтерпретатор Python зустріне перший оператор `return`. Використовуючи цю особливість, можна визначати такі функції:

```
def is_positive(x):
    if x > 0:
        return True
    else:
        return False
```

У цьому випадку ми отримуватимемо значення `True` для додатних чисел і `False` – для від'ємних або нуля. Таку функцію можна використовувати так:

```
positives = []
for a in range(-5, 7):
```

```
    if is_positive(a):
        positives.append(a)
print(positives)      # [1, 2, 3, 4, 5, 6]
```

В результаті, список `positives` міститиме лише додатні числа. Щоправда, в даному випадку, нашу функцію можна реалізувати набагато коротше:

```
def is_positive(x):
    return x > 0
```

Тут сам оператор > повертатиме значення `True` чи `False`.

Якщо потрібно створити функцію, яка приймає два аргументи, наприклад, для обчислення площі прямокутника, це робиться так:

```
def get_area(w, h):
    return w * h
```

Аргументи вказуються через кому, а тіло функції складається всього з одного оператора `return`, в якому відразу виконуються необхідні обчислення.

Викликаємо цю функцію:

```
print(get_area(10, 5.5))    # 55
```

Побачимо результат її роботи – значення `55`. Під час виклику на перше значення `10` посилається перший аргумент `w`, а на значення `5.5` – другий аргумент `h`.

Під час виклику функцій ми маємо передавати рівно стільки аргументів, скільки вказано у визначенні. Наприклад, ось такі виклики не працюватимуть:

```
my_abs ()               # TypeError
my_abs (1, 2)           # TypeError
say_hello ("abc")       # TypeError
```

Тут зазначено або забагато, або замало аргументів.

Однак до будь-якої функції можна додати параметри із дефолтними значеннями:

```
def say(msg, end="!"):
    print(msg + end)
```

І тепер, можна викликати цю функцію так:

```
say("Hello")            # Hello!
```

або так:

```
say("Hello", "!!!")   # Hello!!!
```

Якщо параметр не вказано, береться його дефолтне значення. Якщо ми його явно задаємо, то береться передане значення. Тут слід пам'ятати лише одне правило: параметри із дефолтними значеннями мають бути записані після параметрів без дефолтного значення. Тобто ось такий запис призведе до синтаксичної помилки:

```
def say(end="!",msg):
```

Тепер, давайте додамо до сигнатури нашої функції ще один параметр:

```
def say(msg, end="!", sep=": "):
    print("Message" + sep + msg + end)
```

Викликати цю функцію ми можемо таким чином:

```
say("Hello", "?", " ")
```

Кожному параметру тут відповідатиме своє значення відповідно до зазначеного порядку. А чи можна викликати цю функцію, вказавши лише перший та останній аргумент? Виявляється так, Python дозволяє це зробити:

```
say("Hello", sep=" ")
```

Тут другим аргументом явно вказуємо ім'я параметра і передаємо йому бажане значення, яке буде використано замість дефолтного. В результаті параметри `msg` і `sep` прийматимуть передані значення, а параметр `end` отримає своє дефолтне значення. Параметр, значення якого передається із зазначенням імені цього параметра, називається іменованим.

Якщо нам потрібно відразу повернути кілька значень, це можна зробити так. Припустимо наша функція відразу визначатиме і периметр, і площу прямокутника:

```
def perimeter_and_area(w, h):
    return 2*(w + h), w * h
```

І, далі, викликаємо її:

```
result = perimeter_and_area(2, 3)
print(result)            # (10, 6)
```

Результат можна одразу розпаковувати в окремі змінні:

```
perimeter, area = perimeter_and_area(2, 3)
print(perimeter, area)  # 10, 6
```

В тілі функції можна записувати різні конструкції мови Python. Наприклад, для піднесення числа в цілий степінь можна визначити таку функцію:

```
def my_pow(x, n):
    result = 1
    while n > 0:
        result *= x
        n -= 1
    return result
```

Результат виклику такої функції:

```
print(my_pow(2, 5))    # 32
```

2. У Python функцію можна викликати саму із себе. Це називається **рекурсією**. В ролі прикладу рекурсивної функції наведемо функцію, яка робить те саме, що і щойно наведена `my_pow`, але в інший спосіб:.

```
def pow(x, n):
    if n == 0:
        return 1
    else:
        return x * pow(x, n-1)
```

І далі можна її викликати так:

```
print(pow(2, 3))    # 8
```

Тепер детальніше розберемося, як вона працює. Для початку зауважимо, що

$$x^n = x \cdot x^{n-1}$$

Таким чином, для обчислення значення на поточному n-му кроці достатньо взяти значення на попередньому $(n-1)$-му кроці і помножити його на x.

Запуск функції здійснюється із аргументами 2 та 3: `pow(2, 3)`. Далі виконується тіло функції. Перевіряється перша умова. Вона виявляється хибною, оскільки `3 == 0` дає `False`. Тому йде перехід на блок `else`, і, перш ніж виконати оператор `return`, знову викликається та ж функція `pow`, але вже з аргументами 2 і 2: `pow(2, 2)`.

Виконання функції `pow(2, 3)` призупиняється, і запускається функція `pow(2, 2)`. Знову перевіряється перша умова, яка знову виявляється помилковою, інтерпретатор переходить на блок `else` до оператора `return` і відбувається ще один виклик функції `pow`, але вже з аргументами 2 і 1: `pow(2, 1)`.

Далі знову все повторюється, і, з аналогічних причин, відбувається виклик функції `pow(2, 0)`.

Тепер ситуація змінюється: перша умова стає істинною, і функція `pow(2, 0)` повертає значення 1, і рекурсія не йде далі вглиб. Функція `pow(2, 0)` завершує своє виконання, і керування передається функції `pow(2, 1)`. Але її частково вже виконано. Тому береться значення 1 як результат обчислення `pow(2, 0)`, результат множиться на $x = 2$ і величина 2 повертається функцією `pow(2, 1)`. Управління переходить до функції `pow(2, 2)` і тут ми вже маємо результат множення $x \cdot x$, тобто $2 \cdot 2 = 4$. Нарешті, функція `pow(2, 2)` завершує виконання і передає значення 4 у зовнішню функцію `pow(2, 3)`, яка, у свою чергу, домножує 4 на 2 і повертає остаточний результат 8.

Функції з довільним числом аргументів

Для початку згадаємо, що коли ми говорили про кортежі, то вказували на таку роботу оператора присвоєння:

```
x, y = (1, 2)
```

Тут змінні x та y приймають значення 1 і 2 відповідно. Але коли цих значень виявляється більше, ніж змінних, то виникає помилка:

```
x, y = (1, 2, 3, 4)
```

Якщо поставити оператор *, наприклад перед y, то помилки не виникне:

```
x, *y = (1, 2, 3, 4)
```

Тут змінна `x` буде містити значення 1, а змінна `y` — список `[2, 3, 4]`. Тобто, в другу змінну будуть записані всі значення, що залишилися, у вигляді списку. Або ж, можна зробити так:

```
*x, y = (1, 2, 3, 4)
```

Тут перші три значення будуть перепаковані у список `x = [1, 2, 3]`, а останнє значення потрапляє у змінну `y = 4`. Те ж саме працює і зі списками:

```
x, *y = [1, "a", True, 4]
```

Аналогічно, з рядками:

```
*x, y, z= "Hello world!"
```

Це працює взагалі з будь-якими ітерованими типами даних. Тобто, оператор * упаковує значення, що залишилися, у список.

Цей оператор може виконувати і зворотну операцію – розпаковувати списки в набір даних. Припустимо, лічильник у циклі `for` повинен пробігати діапазон від -5 до 5 з кроком 1:

```
for x in range(-5, 6):
    print(x, end=" ")
```

Далі ми хочемо представити цей діапазон за допомогою списку:

```
a = [-5, 6]
```

та передати його функції `range`. Якщо записати просто:

```
for x in range(a):
```

Те виникне помилка, адже функція чекає на 1, 2 або 3 параметри типу `int` через кому, а не на список. Але якщо поставити * перед змінною `a`, відбудеться розпакування списку у набір із двох чисел: -5 і 6:

```
for x in range(*a):
```

Програма спрацює без помилок. Тоді як дізнатися: коли цей оператор розпаковує дані, а коли запаковує? Дуже просто:

1. Якщо виконується присвоєння даних змінної з оператором *, дані упаковуються. Наприклад, `*y, x = 1,2,3`.
2. Якщо виконується передача списку із зазначенням оператора *, відбувається його розпакування. Наприклад, `range(*[-5,6])`.

Цей оператор можна використовувати для оголошення функцій із довільною кількістю аргументів:

```
def my_func(*args):
    print(args)

my_func()                    # ()
my_func(1)                   # (1,)
my_func(1, 2)                # (1, 2)
my_func(1, 3, "hello")       # (1, 3, "hello")
```

Змінна `args` стає кортежем із переданими значеннями при виклику функції. Отже, ми можемо спокійно перебрати всі значення кортежу, наприклад, у циклі `for`:

```
def my_func(*args):
    for arg in args:
        print(arg)
```

```
my_func(1, 3, "hello")
```

В результаті отримали функцію з довільною кількістю аргументів. Де це може стати в нагоді? Наприклад, реалізувати універсальну функцію для обчислення суми переданих елементів:

```
def my_sum(*args):
    total = 0
    for arg in args:
        total += arg
    return total

print(my_sum(1, 2, 3, 4, 5))    # 15
```

А якщо ми хочемо передавати функції необмежену кількість іменованих аргументів? Наприклад:

```
my_func(arg1=1, arg2=2, arg3=3)
```

У цьому випадку аргумент у функції слід записати з двома зірочками:

```
def my_func(**kwargs):
    print(kwargs)
```

Ім'я `kwargs` – це скорочення від keyword arguments, тобто «іменовані аргументи» і воно посилається на словник, який містить усі передані іменовані аргументи у вигляді пар ім'я – значення. Перебрати елементи словника можна, наприклад, так:

```
def my_func(**kwargs):
    for name, value in kwargs.items():
        print(name, value)
```

Але якщо ми тепер спробуємо викликати функцію ось так:

```
my_func(1, 2, 3, arg1=1, arg2=2, arg3=3)
```

то виникне помилка, адже перші три аргументи не є іменованими. Для такого варіанта дзвінків функцію `my_func` слід записати так:

```
def my_func(*args, **kwargs):
    print(args)
    print(kwargs)
```

Причому, ці неіменовані аргументи повинні завжди йти перед іменованими. Можна дещо ускладнити сигнатуру функції:

```
def my_func(*args, a=10, b="text", **kwargs):
    print(args)
```

```
    print(kwargs)
    print(a, b)
```

Ще варіант:

```
def my_func(x, y, *args, a=10, b="text", **kwargs):
    print(args)
    print(kwargs)
    print(x, y)
    print(a, b)
```

Всі ці комбінації цілком припустимі, головне, щоб їх порядок був саме таким: спочатку позиційні параметри, потім необов'язковий список `args`, потім іменовані параметри, і потім список цих параметрів.

Анонімні або лямбда-функції

У Python існують так звані анонімні функції (лямбда-функції). Що це таке? Давайте спочатку розглянемо такий приклад:

```
def show_elements(lst, func):
    for x in lst:
        if func(x):
            print(x)

def odd(x):
    return x % 2 != 0  # повертає True, якщо x - непарне

numbers=[1, 2, 3, 4, 5, 6, 7]
show_elements(numbers, odd)
```

У нас є функція `show_elements`, яка відображає зі списку lst тільки елементи, для яких функція `func` повертає значення `True`. Далі ми створюємо допоміжну функцію `odd`, яка повертає `True` для непарних значень, після чого йде виклик `show_elements` для списку `numbers` і функції-селектора `odd`.

Але раптом ми захочемо змінити селектор та вибирати, наприклад, усі парні значення? Потрібно створювати нову функцію, а потім вказувати посилання на неї, що здається не дуже зручним. Ось тут нам на допомогу приходять лямбда функції. Їх можна оголосити у будь-якому місці програми із використанням наступного синтаксису:

```
lambda arg1, arg2, …: <вираз>
```

Наприклад:

```
adder = lambda a, b: a + b
```

Цю функцію можна викликати:

```
print(adder(1, 2))
```

В тілі анонімної функції можна писати будь-який вираз, але це має бути саме вираз (конструкція, яка може бути обчислена і зведена до одного об'єкту-значення). На жаль, тут не можна використовувати цикли, розгалуження чи будь-які багаторядкові конструкції. Також ми не пишемо ключове слово return, адже лямбда-функція автоматично повертає результат обчислення тіла-виразу.

У нашому випадку ми можемо записати таку функцію відразу як другий аргумент:

```
show_elements(a, lambda x: x % 2 == 0)
```

Якщо анонімна функція не приймає жодних аргументів, то вона записується так:

```
p = lambda: "hello world"
p()
```

Декоратори

Декоратори – це особливі функції, основним призначенням яких є розширення можливостей інших функцій без зміни коду самої функції. Давайте розглянемо на прикладі, як визначаються та використовуються декоратори. Пригадаємо алгоритм Евкліда для пошуку НСД (**н**айбільший **с**пільний **д**ільник, англ. gcd, **g**reatest **c**ommon **d**ivisor) двох натуральних чисел a та b. Його найпростіша реалізація може виглядати, наприклад, так:

```
def gcd(a, b):
    while a!=b:
        if a > b:
            a-=b
        else:
            b-=a
    return a
```

Далі ми хочемо створити тест для перевірки швидкості роботи цієї функції. Реалізуємо цей тест як декоратора. Це буде виглядати так:

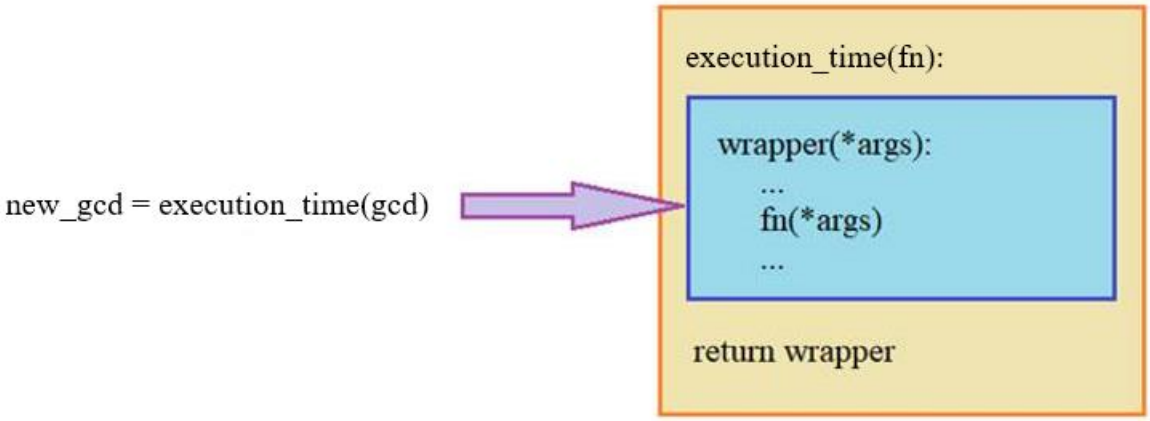
```
from time import time

def execution_time(fn):
    def wrapper(*args, **kwargs):
        start = time()
        result = fn(*args, **kwargs)
        time_elapsed = time() - start
        print(f"Час роботи: {time_elapsed:.3f} с"
        return result
    return wrapper
```

Принцип роботи декоратора можна коротко описати так. Декоратор `execution_time` отримує об'єкт деякої функції `fn`. Всередині тіла декоратора оголошується нова функція-обгортка `wrapper`, яка буде відігравати роль розширеної версії `fn`. Ця функція викликає функцію `fn`, а також реалізує додатковий функціонал (в нашому випадку це вимірювання часу роботи функції `fn` і вивід результату в консоль). Зверніть увагу на параметри обгортки: `*args`, `**kwargs`. Такий набір параметрів здатен приймати довільні аргументи, зберігати їх і передавати далі – у виклик функції `fn` в тілі `wrapper`.

```
new_gcd = execution_time(gcd)
print(new_gcd(450000000, 27))     # Час роботи: 0.692 с
                                  # 9
```

Тобто декоратор `execution_time` повертає посилання на `wrapper`, який у свою чергу викликатиме `gcd` і визначатиме час виконання саме цієї функції при заданих аргументах.



Відзначимо ще одну особливість створеного декоратора: обгортка `wrapper` використовує аргумент `fn` зовнішньої функції `execution_time`. Коли ми робимо виклик

```
new_gcd = execution_time(gcd)
```

у нас створюються два об'єкти-функції: `execution_time` і `wrapper`. На `wrapper` посилається глобальна змінна `new_gcd`, а сам `wrapper` містить посилання зовнішній контекст, тобто вміст функції `execution_time`, звідки і бере змінну `fn`. Завдяки наявності цього посилання об'єкт `execution_time` не видаляється збирачем сміття і продовжує існувати, доки існує `wrapper`. Це у програмуванні називається замиканням.

Отже, **замикання** — це функція, яка визначена в тілі іншої, зовнішньої функції, і посилається на змінні, визначені в зовнішній функції.

Тепер, коли ми детально розібралися з роботою декоратора, викличемо його для швидкого алгоритму Евкліда. Він реалізується таким чином:

```
def gcd_fast(a, b):
    if a < b: a , b = b , a
    while b:
        a, b = b , a % b
    return a
```

І далі, запишемо:

```
new_gcd = execution_time(gcd)
new_gcd_fast = execurtion_time(gcd_fast)

new_gcd(450000000, 27)          # Час роботи: 0.692 с
new_gcd_fast(450000000, 27)     # Час роботи: 0.000 с
```

Як ми вже бачили, для того, щоб використовувати декоратор, нам потрібно створити нову функцію, яка має збагачений функціонал і є результатом застосування декоратору до початкової функції. Ми робили це в рядку

```
new_gcd = execution_time(gcd)
```

Дуже часто не створюють нове ім’я, а просто оновлюють старе:

```
gcd = execution_time(gcd)
```

Така операція є дуже вживаною у мові Python, а тому для неї є спеціальний синтаксис. Така підміна функції на її декоровану версію відбувається ще на етапі визначення, тобто біля інструкції def:

```
@execution_time
def gcd(a, b):
    while a!=b:
        if a > b:
            a-=b
        else:
            b-=a
    return a
```

В результаті ми отримаємо функцію `gcd`, яка вимірює і друкує час свого виконання.

Декоратор `execution_time`, який ми створювали, є дуже спрощеним порівняно з тими декораторами, які використовуються на практиці. Наведемо загальний шаблон написання довільного декоратора:

```
from functools import wraps

# створення декоратора
def decorator(fn):
    @wraps(fn)   # зберігає атрибути функції
    def wrapper(*args, **kwargs):
        # робота декоратора до виклику основної функції
        result = fn(*args, **kwargs)
        # робота декоратора після виклику осн. функції
        return result
    return wrapper

# використання декоратора
@decorator
def function(…):
    …
```

Завдання:

Написати програми у середовищі PyCharm відповідно до номера свого варіанту (кожне завдання оцінюється в 2 бали, максимум 8 балів за роботу).

Варіанти:

Варіант №1

Завдання №1. Оголосіть функцію, яка приймає три сторони трикутника та перевіряє, чи можна з перерахованих аргументів скласти трикутник.
(у довільного трикутника довжина будь-якої сторони має бути меншою від суми двох інших, це називається *нерівністю трикутника*). Якщо перевірка проходить, повернути значення `True`, інакше значення `False`. Викличте функцію та виведіть на екран ТАК, якщо функція поверне `True` та НІ, якщо повертається значення `False`.

Завдання №2. Вводиться ціле додатне число N. Необхідно написати рекурсивну функцію, яка відображає на екрані послідовність цілих чисел від 1 до N (включно). Кожне число виводиться з нового рядка.

Завдання №3. Визначте функцію-замикання, яка збільшувала б значення переданого аргументу `x` на величину `summand` (параметр зовнішньої функції, ціле число) і повертала б обчислений результат. Зовнішня функція має приймати `summand` як аргумент і повертати замикання. З клавіатури зчитайте значення `summand` (на окремому рядку) і список аргументів `x` (усі значення – цілі числа, розділені пробілами). Потім створіть замикання, збережіть посилання на нього в імені `adder`. Застосуйте функцію `adder` до всіх значень `x` і виведіть результат на екран у вигляді списку.

Завдання №4. Оголосіть функцію з ім'ям `get_rectangle_area`, яка обчислює площу прямокутника за двома параметрами: `width` і `height` (шириною та висотою прямокутника) та повертає результат. Створіть декоратор `auto_print`, який відображує результат роботи декорованої функції у вигляді рядка «Результат обчислень: …". Викличте декоровану функцію `get_rectangle_area` на значеннях, які вводить користувач.

Варіант №2.

Завдання №1. Розглянемо деяке число n. Якщо воно парне — поділимо його на 2. А якщо непарне, то обчислимо значення $3n + 1$. До отриманого числа застосуємо ту саму логіку. Таким чином отримується послідовність чисел. Гіпотеза Коллатца полягає у тому, що яке б число n ми не взяли, рани чи пізно ми отримаємо 1. Наприклад, якщо $n = 13$, то матимемо послідовність:

$$13 \rightarrow 40 \rightarrow 20 \rightarrow 10 \rightarrow 5 \rightarrow 16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$$

Створіть функцію, яка приймає число n як аргумент і повертає список, який містить утворену послідовність. Користувач вводить число n, і програма

друкує послідовність на екрані. До речі, гіпотеза Коллатца досі залишається недоведеною в математиці.

Завдання №2. Користувач вводить рядок натуральних чисел через пробіл. Перетворіть цей рядок на кортеж чисел. Використовуючи вбудовану функцію max і її іменований параметр `key`, знайдіть у цьому кортежі число, в якому буде максимальним відношення суми цифр до кількості цифр. Ключ задайте за допомогою lambda-функції. Результат надрукувати на екрані у вигляді списку. Наприклад, для вхідних даних

```
23 4534 564 324 654 1223 756 54 546 23 456
```

результат має виглядати так:

```
756
```

адже величина $\frac{7+5+6}{3} = 6$ – найбільша серед усіх наявних чисел.

Завдання №3. Створіть функцію-замикання, яка розміщує переданий їй рядок `text` (параметр внутрішньої функції) у довільний тег, що міститься в змінній `tag` – параметрі зовнішньої функції. Наприклад, для рядку «Ой, у лузі червона калина» і тегу «b» результат має виглядати так:

```
<b>Ой, у лузі червона калина</b>
```

Далі, на вхід програми надходять два рядки: перший з тегом, другий з деяким текстом. Другий рядок слід помістити в тег з першого рядка за допомогою реалізованого замикання. Результат виведіть на екран.

Завдання №4. На вхід програми надходить рядок із цілих чисел, записаних через пробіл. Напишіть функцію `get_list`, яка перетворює цей рядок на список із цілих чисел і повертає його. Визначте декоратор `get_sorted` для цієї функції, який сортує список чисел, отриманий із функції, що декорується. Викличте декоровану функцію `get_list` і відобразіть отриманий відсортований список на екрані.

Варіант №3

Завдання №1. Оголосіть функцію перевірки числа на непарність (повертається True, якщо передане число непарне і False, якщо число парне). Після оголошення функції прочитайте список цілих значень, записаних в один рядок через пробіл. І, використовуючи генератор списків та оголошену функцію, сформуйте список із непарних значень на основі введеного вихідного списку. Результат виведіть на екран.

Завдання №2. З клавіатури вводиться натуральне число n. Необхідно за допомогою рекурсивної функції сформувати послідовність чисел Фібоначчі за правилом: перші два числа дорівнюють 1 і 1, а кожне наступне значення дорівнює сумі двох попередніх. Приклад такої послідовності для перших 7

чисел: 1, 1, 2, 3, 5, 8, 13... Функція повинна повертати список сформованої послідовності довжиною n. Викличте цю функцію та відобразіть на екрані отриманий список.

Завдання №3. Використовуючи замикання функцій, оголосіть внутрішню функцію, яка приймає два рядки-параметри: ім'я та прізвище і повертає рядок, який має бути сформований за шаблоном. Шаблон – це рядок, який передається зовнішньої функції і, наприклад, може мати такий вигляд: `"Шановний %first_name% %last_name%! Ви робите замикання функцій."` Тут `%first_name%` і `%last_name%` — це фрагменти куди замикання підставляє передане йому ім'я і прізвище відповідно. Шаблон може бути іншим, ви це визначаєте самі, але він має містити `%first_name%` і `%last_name%`. Тут важливо, щоб внутрішня функція вміла підставляти дані у шаблон, формувати новий рядок і повертати результат. Викличте замикання та відобразіть на екрані результат її роботи для імені і прізвища, яке введе користувач.

Завдання №4. Користувач вводить два рядки, кожен з яких містить час у форматі HH:MM:SS, причому обидва моменти часу належать одній добі. Створити функцію, яка приймає ці два рядки і повертає різницю між вказаними моментами часу в секундах. Створити декоратор для цієї функції, який буде перетворювати різницю часів у секундах на рядок формату HH:MM:SS. Наприклад, якщо введено часи 23:30:27 і 20:19:14, то недекорована функція має повернути число 11473, а декорована — рядок `"03:11:13"`

Варіант №4

Завдання №1. Користувач вводить слово і два числа в один рядок через пробіл. Якщо слово — `"Area"`, слід оголосити функцію з ім'ям `get_rectangle_property`, яка приймає два аргументи `width` і `height`, і обчислює площу прямокутника з відповідними розмірами, а також повертає обчислене значення. Якщо введене слово не `"Area"` (будь-яке інше), то оголошується функція з тим самим ім'ям `get_rectangle_property` і такими ж параметрами, але обчислювати вона має вже периметр прямокутника. Обчислене значення повертається функцією. Викличте створену функцію з двома наданими аргументами та виведіть результат на екран.

Завдання №2. Користувач вводить натуральне число n. Необхідно за допомогою рекурсивної функції обчислити *подвійний факторіал* числа n. Подвійний факторіал числа n позначається $n!!$ і дорівнює добутку всіх натуральних чисел, менших або рівних n, які мають ту ж парність, що і число

n. Наприклад, $7!! = 1 \cdot 3 \cdot 5 \cdot 7 = 105$, $8!! = 2 \cdot 4 \cdot 6 \cdot 8 = 384$. Рекурсивна функція має повертати обчислене значення. Викличте цю функцію та відобразіть на екрані результат її роботи.

Завдання №3. Використовуючи замикання функцій, створіть внутрішню функцію, яка приймає один аргумент (натуральне число) і повертає кількість n-цифрових аргументів, які були передані за весь час її існування. Число n передається зовнішній функції як аргумент. Спочатку користувач вводить число n. Після цього програма пропонує вводити числа по одному, натискаючи "Enter" після кожного введеного числа. На кожне введене число користувач отримує відповідь: кількість n-цифрових чисел, які були введені до цього моменту. Кожне введене число передається аргументом замиканню і результат виводиться на екран. Програма припиняє запити, якщо користувач вводить порожній рядок. Наприклад:

```
Введіть n: 2
Вводіть числа по одному, натискаючи "Enter" після кожного
числа:
45
Було введено 2-цифрових чисел: 1
245
Було введено 2-цифрових чисел: 1
98
Було введено 2-цифрових чисел: 2
1
Було введено 2-цифрових чисел: 2
```

і т.д.

Завдання №4. Створіть рекурсивну функцію, яка обчислює n-не число Фібоначчі (F_n). $F_1 = 1$, $F_2 = 1$, а кожне наступне дорівнює сумі двох попередніх: $F_n = F_{n-1} + F_{n-2}$. Далі створіть декоратор `memoize` який би зберігав обчислені значення у словнику (ключ – n, а значення – F_n). Далі, коли йде виклик з аргументом n, декоратор перевіряє, чи було вже обчислене значення числа Фібоначчі F_n. Якщо ні, то декоратор запускає основну функцію і обчислює його, після чого зберігає у словнику. Якщо ж таке значення вже було обчислене (а, отже, є у словнику), то він просто повертає збережене значення без обчислень.

Варіант №5

Завдання №1. Оголосіть функцію, яка приймає рядок (як аргумент) і повертає `False`, якщо довжина рядка менше трьох символів. Інакше повертається

значення `True`. Після оголошення функції прочитайте список назв міст, записаних в один рядок через пробіл. Потім, використовуючи генератор списку та створену функцію, сформуйте список із назв міст довжиною не менше трьох символів на основі введеного вихідного списку. Результат надрукуйте на екрані.

Завдання №2. Є наступний багатовимірний список: d = [1, 2, [True, False], ["Київ", "Харків", [100, 101], [True, [1, -1]]], 7.89]. За допомогою рекурсивної функції створити на його основі одновимірний список значень елементів списку d. Функція повинна повертати новий створений одновимірний список. Викличте цю функцію та відобразіть на екрані результат її роботи. Функція має працювати з будь-яким подібним багатовимірним списком.

Завдання №3. Використовуючи замикання функцій, оголосіть внутрішню функцію, яка перетворює рядок зі списку цілих чисел, записаних через пробіл, або на список, або на кортеж цілих чисел. Тип колекції визначається параметром `collection_type` зовнішньої функції. Якщо `collection_type == "list"`, використовується список, інакше – кортеж. Далі, на вхід програми надходить два рядки: перший – це значення для параметра type; друга – список цілих чисел, записаних через пробіл. За допомогою реалізованого замикання перетворити цей рядок на відповідну колекцію. Результат замикання виведіть на екран.

Завдання №4. Оголосіть функцію з ім'ям `transliterate`, яка приймає рядок кирилицею та перетворює його на рядок латиницею, використовуючи словник транслітерації, наприклад, звідси http://ukrlit.org/transliteratsiia. Функція повинна повертати перетворений рядок. Заміни робити без урахування регістру (початковий рядок перевести в нижній регістр – малі літери). Усі небуквенні символи `"!?:;.,_"` перетворювати на символ `'-'` (дефісу). Створіть декоратор для цієї функції, який модифікує результат роботи функції `transliterate` таким чином: якщо в рядку зустрічається кілька дефісів, що йдуть поспіль, вони замінюються на один дефіс. Задекоруйте функцію `transliterate`. Викличте утворену функцію на рядку, який вводить користувач і виведіть результат на екран.

Варіант №6

Завдання №1. Оголосіть функцію, яка приймає рядок (як аргумент) і повертає два значення у вигляді кортежу: переданий рядок та його довжина. Після оголошення функції прочитайте список назв міст, записаних в один рядок через пробіл. Потім, використовуючи генератор словників та створену

функцію, сформуйте словник `dictionary` у форматі: `dictionary` = {<місто 1>: <число символів>, ..., <місто N>: <число символів>}

Завдання №2. Користувач вводить рядок натуральних чисел через пробіл. Перетворіть цей рядок на список чисел. Використовуючи вбудовану функцію `sorted` і її іменований параметр `key`, відсортуйте ці числа наступним чином: числа мають йти у порядку зростання останньої цифри (розряд одиниць), а серед чисел з однаковою останньою цифрою раніше мають йти числа з більшою кількістю цифр. Ключ задайте за допомогою lambda-функції. Результат надрукувати на екрані у вигляді списку. Наприклад, для вхідних даних

```
45 9837 345 787 3 3925 34 13
```

результат має виглядати так:

```
[13, 3, 34, 3925, 345, 45, 9837, 787]
```

Завдання №3. Використовуючи замикання функцій, оголосіть внутрішню функцію, яка приймає два параметри a і b, після чого повертає рядок у форматі:

```
"<f>(<a>, <b>) = <число>"
```

Наприклад, якщо функція – sum, а числа a і b дорівнюють 3 і 4 відповідно, то рядок, сформований замиканням має виглядати так:

```
"sum(3, 4) = 7"
```

де <число> це обчислене значення функції f; <a>, <b> — відповідні значення a і b, а <f> — справжня назва функції (її можна отримати як `f.__name__`). Посилання на f передається як аргумент зовнішньої функції. Викличте замикання та відобразіть на екрані результат його роботи. Функцію f придумайте самостійно (не використовуйте вбудовані функції), дайте їй хорошу назву і реалізуйте (вона повинна робити щось з двома параметрами a, b і повертати результат).

Завдання №4. Оголосіть функцію, яка приймає колекцію (список, кортеж) довжин сторін багатокутника і обчислює периметр цього багатокутника. Ця функція має повертати кортеж з двох значень — кількості кутів багатокутника і периметру. Створіть декоратор для цієї функції, який виводить на екран повідомлення: `"Периметр <фігура> дорівнює <число>"`, де <число> — результат роботи нашої функції, а <фігура> — це слово `"трикутника"`, `"чотирикутника"`, тощо. Якщо кутів більше за 8, то писати `"9-тикутника"`, `"10-тикутника"` тощо. Застосуйте декоратор до функції та викличте декоровану функцію на списку сторін, який передав користувач.

Варіант №7

Завдання №1. Оголосіть функцію перевірки числа на парність (повертається `True`, якщо передане число парне і `False`, якщо число непарне). Після

оголошення функції у циклі необхідно зчитувати ціле числове значення (функцією `input`), доки надійде число 1. Якщо введене значення парне (перевіряється за допомогою заданої функції), воно виводиться на екран.

Завдання №2. Оголосіть звичайну функцію, яка першим параметром приймає список, а другим – посилання на функцію. Вона має повертати фільтрований список. Робиться це за таким алгоритмом. У циклі переберіть елементи списку, для кожного викличте, передану другим параметром функцію і, якщо вона повертає `True`, то елемент повинен бути доданий до списку, що повертається основною функцією. Викличте оголошену функцію, першим параметром передайте список чисел, які ввів користувач, а другим запишіть лямбда-функцію для відбору елементів, які є квадратами натуральних чисел. Результат роботи функції виведіть на екран.

Завдання №3. Використовуючи замикання функцій, оголосіть внутрішню функцію, яка на основі двох параметрів обчислює і повертає площу фігури. Якої саме фігури: трикутника чи прямокутника, визначається параметром `type` зовнішньої функції. Якщо `type` набуває значення `"Triangle"`, то обчислюється площа трикутника (тоді параметри — це основа і висота), інакше – прямокутника (тоді параметри — це ширина і висота). Параметр `type` має дефолтне значення `"Triangle"`. Користувач вводить значення type і два параметри в один рядок через пробіл. За допомогою створеного замикання обчисліть площу й відобразіть на екрані результат.

Завдання №4. Вводяться два списки (кожен із нового рядка) зі слів, записаних через пробіл. Створіть функцію, яка перетворює ці два рядки на два списки слів і повертає ці списки (кортежом списків). Визначте декоратор для цієї функції, який із цих двох списків формує словник, у якому ключами є слова з першого списку, а значення – відповідні елементи з другого списку. Застосуйте декоратор до функції та викличте її. Результат (словник) відобразіть на екрані.

Варіант №8

Завдання №1. Напишіть функцію, яка перевіряє коректність переданої їй email-адреси у вигляді рядка. Вважатимемо, що адреса вірна, якщо вона обов'язково містить символи `'@'` і `'.'` в такій послідовності, і ці символи не є першими, останніми і не стоять поруч. Інші символи можуть приймати значення: `'a'-'z'`, `'A'-'Z'`, `'0'-'9'` і `'_'`. Якщо email вірний, то функція повертає `True`, в протилежному випадку — `False`. Після оголошення функції прочитайте рядок з email-адресою (за допомогою функції

input) та викличте функцію з цим аргументом. Результат надрукуйте на екрані. Не використовувати регулярні вирази (модуль `re`).

Завдання №2. Вводиться список цілих чисел в один рядок через пробіл. Необхідно обчислити суму введених значень, використовуючи рекурсивну функцію (для перебору елементів списку). Функція має повертати значення суми. (Виводити на екран вона нічого не повинна). Викликати рекурсивну функцію для введеного з клавіатури списку та відобразити значення обчисленої суми на екрані.

Завдання №3. Використовуючи замикання функцій, оголосіть внутрішню функцію, яка б замінювала деякі символи на інші символи. Які символи шукати і на які замінювати, визначається параметром зовнішньої функції `replace_dict`. Це словник, ключами якого є символи, які ми хочемо замінити, а значення — це, відповідно, символи, на які треба замінювати. Внутрішній функції передається лише рядок для перетворення. Перетворений рядок має повертатися внутрішньою функцією. Викличте отримане замикання на рядку, який ввів користувач, і відобразіть на екрані результат її роботи.

Завдання №4. Оголосіть функцію, яка обчислює площу кола та повертає обчислене значення. Як аргумент їй передається значення радіуса.

Визначте декоратор для цієї функції, який виводить на екран повідомлення:

`"Площа кола дорівнює <число>"`

Тут <число> — результат обчислень з точністю до сотих. Застосуйте декоратор до функції та викличте декоровану функцію на радіусі, який ввів користувач.

Варіант №9

Завдання №1. Оголосіть функцію, яка приймає список, знаходить максимальне, мінімальне та суму значень цього списку та повертає всі обчислені результати у вигляді кортежу. Викличте оголошену функцію зі списком чисел, введеними з клавіатури в один рядок через пробіл, і відобразіть результат роботи функції на екрані.

Завдання №2. За допомогою рекурсивної функції знайдіть мінімальне значення елементів переданого їй кортежу. Функція повинна повертати мінімальне значення. Користувач вводить рядок чисел через пробіл. Перетворіть ці дані у кортеж чисел і передайте його рекурсивній функції. Результат (мінімальне значення) відобразіть на екрані.

Завдання №3. Використовуючи замикання функцій, оголосіть внутрішню функцію, яка приймає як аргумент колекцію (список або кортеж) і повертає або мінімальне значення, або максимальне, залежно від значення параметра `mode` зовнішньої функції. Якщо `mode` дорівнює `"max"`, то повертається

максимальне значення, інакше мінімальне. Параметр `mode` має дефолтне значення `"max"`. Користувач вводить значення параметру `mode` і список чисел в один рядок через пробіл. Викличте замикання на введених даних та відобразіть на екрані результат його роботи.

Завдання 4. Оголосіть функцію, яка приймає рядок із кирилицею та перетворює його на рядок латиницею, використовуючи словник для заміни українських букв на відповідне латинське написання (http://ukrlit.org/transliteratsiia). Функція повинна повертати перетворений рядок. Заміни робити без урахування регістру (початковий рядок перевести в нижній регістр – малі літери). Визначте декоратор, який символи `"?!:;,."` перетворює на символ `"-"` і, крім того, всі дефіси, що йдуть підряд (наприклад, `"--"` або `"---"`) призводить до одного дефісу. Отриманий результат має повертатися у вигляді рядка. Викличте декоровану функцію на рядку, який ввів користувач. Результат надрукуйте на екрані.

Варіант №10

Завдання №1. Запишіть функцію без параметрів, яка зчитує з клавіатури прізвище, ім'я та по батькові, записані в один рядок через пробіл, і повертає три рядки: прізвище, ім'я та по батькові (у вигляді кортежу). Викличте оголошену функцію та відобразіть на екрані рядок у форматі:

`"Шановний <Ім'я> <Прізвище>! Ви правильно виконали цю роботу!"`

Завдання №2. За допомогою рекурсивної функції сформуйте список із відфільтрованих значень переданого їй кортежу. Критерій фільтрації визначити за допомогою анонімної функції, що передається рекурсивній функції другим аргументом (першим аргументом передається кортеж). Викличте цю функцію та відобразіть результат її роботи на екрані. Функцію-фільтр вигадайте самостійно. Вона має приймати одне число і повертати `True` чи `False` в залежності від того, чи задовольняє це число критерію (наприклад, чи є це число парним).

Завдання №3. Використовуючи замикання функцій, оголосіть внутрішню функцію, яка приймає як аргумент список цілих чисел і видаляє з нього всі парні чи непарні значення, залежно від значення параметра `mode`. Якщо `mode` дорівнює `"even"`, то видаляються парні значення, інакше – непарні. Параметр `type` має дефолтне значення `"even"`. Користувач вводить значення параметру `mode` і список чисел в один рядок через пробіл. Викличте замикання на введених даних та відобразіть на екрані результат його роботи.

Завдання №4. Оголосіть функцію, яка приймає рядок, видаляє з неї всі пробіли, що йдуть поспіль, і переводить її в нижній регістр – малі літери. Результат (рядок) повертається функцією. Визначте декоратор, який рядок, повернутий функцією, перетворює на код Морзе, використовуючи словник для заміни англійських літер на відповідні послідовності з точок і тире. Абетку Морзе можна знайти у додатку нижче. Перетворений рядок повертається декоратором. Застосуйте декоратор до функції та викличте декоровану функцію. Результат роботи надрукуйте на екрані.

Додаток:

1. Словник для кодування кирилиць і символу пробілу азбукою Морзе:

```
morze = {'A':'.-', 'B':'-...', 'C':'-.-.', 'D':'-..', 'E':'.', 'F':'..-.', 'G':'--.', 'H':'....', 'I':'..', 'J':'.---', 'K':'-.-', 'L':'.-..', 'M':'--', 'N':'-.', 'O':'---', 'P':'.--.', 'Q':'--.-', 'R':'.-.', 'S':'...', 'T':'-', 'U':'..-', 'V':'...-', 'W':'.--', 'X':'-..-', 'Y':'-.--', 'Z':'--..'}
```

Зміст звіту

1. Титульний лист із назвою лабораторної роботи, номером свого варіанту, прізвищем студента та групи.
2. Мета лабораторної роботи.
3. Завдання.
4. Лістинг програми (скопійований з IDE).
5. Результати роботи програми (скріншоти).